

Distributed Systems – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 8 Jan 2027 - 21 Jan 2027 (Days 145-158)

 **Your slots (IST):** Mon-Fri **7:30-8:30pm** & **9:00-10:00pm** · Sat-Sun **9:00am-12:00pm, 3:00-5:30pm, 6:00-8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Deep dive into CAP, consensus, replication, partitioning, transactions, clocks, failure modes, and 80 LPA interview preparation.

Target Level: Google L5, Meta E4, Amazon SDE3, Microsoft L63+ (80+ LPA TC in India)

PREREQUISITES – Complete these PDFs FIRST (do not skip)

Order	PDF	Why required
1	<code>10b-networking-http-fundamentals-complete.pdf</code>	TCP, HTTP, DNS context
2	<code>06-databases-complete.pdf</code>	Replication, transactions base
3	<code>10c-system-design-prerequisites-complete.pdf</code>	Scaling, caching, load balancers
4	<code>10-system-design-complete.pdf</code>	HLD framework before distributed deep dive

When to study: Month 6, Days 151-158 (after PDF #10c and #10).

Gate: Explain load balancer + read replica before opening this file.

Table of Contents

1. [CAP Theorem Deep Dive](#)
2. [PACELC Theorem](#)
3. [Consistency Models](#)
4. [Consensus: Raft, Paxos, Leader Election](#)
5. [Replication Strategies](#)
6. [Partitioning & Sharding](#)
7. [Distributed Transactions](#)
8. [Idempotency & Exactly-Once Delivery](#)
9. [Clocks: Physical, Logical, Vector](#)
10. [Failure Modes](#)
11. [Interview Q&A \(20 Questions\)](#)
12. [Where to Practice](#)

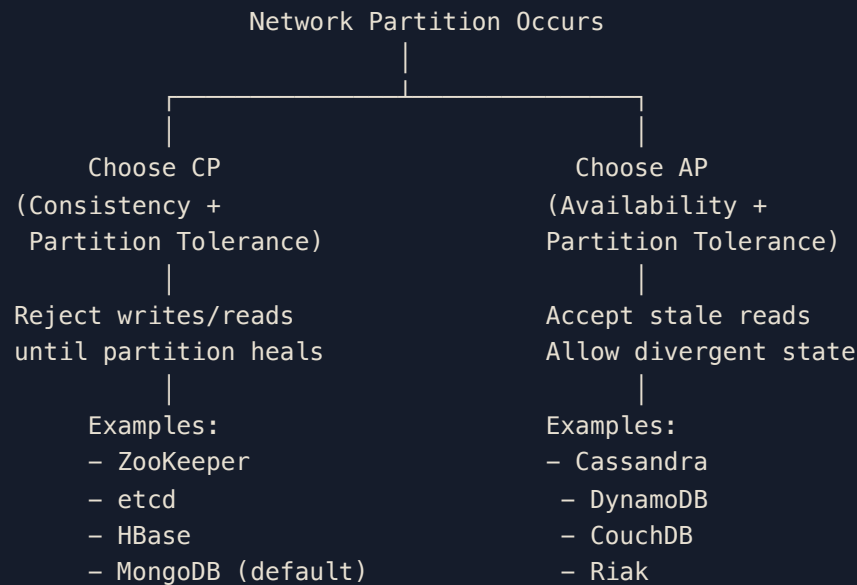
1. CAP Theorem Deep Dive

The Three Properties

Property	Definition	Real-World Meaning
Consistency (C)	Every read receives the most recent write or an error	All nodes see the same data at the same time
Availability (A)	Every request receives a non-error response (not necessarily the latest)	System always responds, even during failures
Partition Tolerance (P)	System continues operating despite network partitions	Messages can be lost or delayed between nodes

The Fundamental Trade-off

In a distributed system, **network partitions are inevitable** (P is non-negotiable in practice). When a partition occurs, you must choose between C and A:



CAP Is Not a Binary Choice

Modern systems operate on a **continuum**:

- **MongoDB**: Configurable — write concern **majority** = CP; **w:1** = AP-ish
- **Cassandra**: Tunable consistency — **QUORUM** reads/writes = CP; **ONE** = AP
- **PostgreSQL** with synchronous replication: CP
- **PostgreSQL** with async replication: AP (reads may be stale on replicas)

Common Misconceptions

Misconception	Reality
"Pick 2 of 3 always"	P is mandatory in distributed systems; you choose between C and A during a partition
"CA systems exist"	Single-node databases are CA; multi-node systems must handle partitions
"CAP applies to every operation"	Consistency and availability trade-offs can vary per operation (tunable)

2. PACELC Theorem

Beyond CAP

PACELC (Daniel Abadi, 2012) extends CAP to the normal (non-partition) case:

If Partition (P), choose between Availability (A) and Consistency (C). Else (E), choose between Latency (L) and Consistency (C).

PACELC Classification:

PA/EL → Dynamo, Cassandra, Riak
Available during partition; low latency when no partition (eventual consistency)

PC/EC → BigTable, HBase, MongoDB (strong mode)
Consistent during partition; consistent when no partition (higher latency)

PC/EL → Rare – consistent always but low latency when no partition
(Hard to achieve; most systems sacrifice one)

PA/EC → Also rare

Why PACELC Matters for Interviews

At 80 LPA level, interviewers expect you to explain **why** DynamoDB is fast (EL) but may return stale reads, while Spanner is consistent (EC) but has higher write latency due to TrueTime synchronization.

System	PACELC	Write Latency	Consistency
DynamoDB	PA/EL	~single-digit ms	Eventual (default)
Cassandra	PA/EL	Low	Tunable
Google Spanner	PC/EC	Higher (global)	External consistency
CockroachDB	PC/EC	Medium	Serializable
etcd	PC/EC	Medium	Linearizable

3. Consistency Models

Consistency Spectrum (Strongest → Weakest)

```
Linearizability (Strongest)
  ↓
Sequential Consistency
  ↓
Causal Consistency
  ↓
Eventual Consistency (Weakest)
```

Strong Consistency (Linearizability)

Definition: Every operation appears to take effect atomically at some point between its invocation and response. All nodes agree on a single total order of operations.

Properties:

- Reads always return the latest committed write
- Writes are immediately visible to all subsequent reads
- Equivalent to a single-copy semantics

Implementation:

- Single leader with synchronous replication (Raft, Paxos)
- Quorum reads/writes ($R + W > N$)
- Google Spanner uses TrueTime + two-phase commit

Trade-off: Higher latency, lower availability during partitions

```
// Strong consistency example: MongoDB with majority write concern
await collection.updateOne(
  { _id: userId },
  { $set: { balance: newBalance } },
  { writeConcern: { w: 'majority', wtimeout: 5000 } }
);
// Subsequent read from any node with readConcern 'majority' sees this write
```

Eventual Consistency

Definition: If no new updates are made to a given data item, eventually all accesses will return the last updated value.

Properties:

- No guarantee on when convergence happens
- Reads may return stale data indefinitely (without anti-entropy)
- Conflicts may occur and need resolution (LWW, CRDTs, vector clocks)

Use Cases:

- Social media likes/follower counts
- DNS propagation
- CDN cache invalidation
- Shopping cart (Amazon's famous example)

Conflict Resolution Strategies:

Strategy	Description	Example
Last-Write-Wins (LWW)	Timestamp-based; latest write wins	DynamoDB default
Version Vectors	Track causality; detect conflicts	Riak
CRDTs	Mathematically merge without coordination	Collaborative editing
Application-level merge	Custom logic for business rules	Shopping cart merge

Causal Consistency

Definition: Operations that are causally related are seen by all processes in the same order. Concurrent operations may be seen in different orders.

Causal Relationships:

- If $A \rightarrow B$ (A happened before B), everyone sees A before B
- If A and B are concurrent, order doesn't matter

Implementation:

- Vector clocks track causality

- Lamport timestamps provide partial ordering

```
Process P1: write(x=1)      _____→ t=1
Process P2:      read(x=1) → write(x=2)  t=2 (causally after P1)
Process P3: write(y=3)      _____→ t=1 (concurrent with P1)
```

Causal guarantee: All nodes see x=1 before x=2
No guarantee: Order of x=1 and y=3 across nodes

Read Your Writes Consistency

Weaker than strong but stronger than eventual:

- A process always sees its own writes
- Other processes may see stale data
- Implemented via session stickiness or version tracking per client

4. Consensus: Raft, Paxos, Leader Election

Why Consensus Matters

Distributed systems need agreement on:

- Who is the leader?
- What is the committed log entry?
- What is the cluster membership?

Without consensus → split brain, data loss, inconsistent state.

Raft (Understand This Deeply)

Raft decomposes consensus into three sub-problems:

1. Leader Election

States: Follower → Candidate → Leader

Timing:

- Election timeout: 150–300ms (randomized)
- Heartbeat interval: 50ms

Election Process:

1. Follower times out (no heartbeat from leader)
2. Becomes Candidate, increments term, votes for self
3. Sends RequestVote RPCs to all peers
4. If majority votes → becomes Leader
5. If another leader discovered → revert to Follower
6. If split vote → new election after timeout

Safety Properties:

- At most one leader per term
- Leader must have all committed entries
- Committed entries are never lost

2. Log Replication

Client Request → Leader → Append to local log → Replicate to followers

↓
Majority acknowledge
↓
Commit entry → Apply to state machine
↓
Respond to client

Log Entry: [term, index, command]

Commit Rule: Entry committed when replicated on majority of servers

3. Safety (Log Matching)

- If two entries have same index and term → same command
- If entry is committed in a term, it appears in all higher-term leaders' logs

Paxos Simplified

Paxos is the theoretical foundation; Raft is more practical.

Roles:

- **Proposers:** Propose values
- **Acceptors:** Vote on proposals
- **Learners:** Learn chosen values

Two Phases:

Phase 1 (Prepare):

Proposer → Acceptors: Prepare(n) [n = proposal number]

Acceptors → Proposer: Promise(n, accepted_n, accepted_value)

Phase 2 (Accept):

Proposer → Acceptors: Accept(n, value)

Acceptors → Proposer: Accepted(n)

If majority accepts → value is chosen

Multi-Paxos Optimization: Elect a stable leader (like Raft) to skip Phase 1 for most operations.

Leader Election Patterns

Pattern	Used In	Mechanism
Bully Algorithm	Simple clusters	Highest-ID node wins
Ring Algorithm	Token-based systems	Pass token around ring
Raft Election	etcd, Consul	Randomized timeout + voting
ZooKeeper	Kafka (older)	Sequential ephemeral nodes
K8s Lease	Kubernetes	Lease object with TTL

etcd / Raft in Production

etcd cluster (3 or 5 nodes):

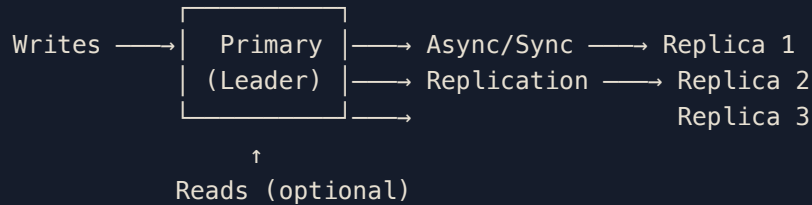
- 3 nodes: Tolerates 1 failure
- 5 nodes: Tolerates 2 failures
- Always use odd numbers (avoid split brain in even clusters)

Write path: Client → Leader → Replicate → Commit → Response

Read path: Client → Any node (with linearizable read option: go through leader)

5. Replication Strategies

Primary-Replica (Leader-Follower)



Pros:

- Simple mental model
- Strong consistency possible (sync replication)
- No write conflicts

Cons:

- Single write bottleneck
- Failover complexity (promote replica, reconfigure clients)
- Leader failure = write unavailability until election

Sync vs Async Replication:

Mode	Durability	Latency	Availability
Sync	High (no data loss)	High	Lower (wait for replicas)
Async	Risk of data loss	Low	Higher
Semi-sync	Compromise (1 sync replica)	Medium	Medium

Multi-Leader (Multi-Master)



Pros:

- Write to nearest leader (lower latency globally)
- Higher write availability (regional leaders)

Cons:

- Write conflicts inevitable (same key updated in two regions)
- Conflict resolution required (LWW, CRDTs, custom merge)
- Complex to reason about

Use Cases: Multi-region deployments (CockroachDB geo-partitioning, Cassandra multi-DC)

Leaderless (Peer-to-Peer)



Any node accepts reads/writes
Quorum: $W + R > N$ for strong consistency

Dynamo-style Quorum:

- N = replication factor (typically 3)
- W = write quorum (typically 2)
- R = read quorum (typically 2)
- $W + R > N$ ensures read sees latest write

Pros:

- No single point of failure
- High availability
- Tunable consistency

Cons:

- Read repair needed for stale replicas
- Sloppy quorum for hinted handoff during failures
- Eventual consistency by default

Hinted Handoff: When a node is down, another node temporarily stores writes and forwards when the node returns.

6. Partitioning & Sharding

Why Partition?

- Single node can't hold all data (storage limit)
- Single node can't serve all traffic (throughput limit)
- Partitioning enables horizontal scaling

Partitioning Strategies

Range-Based Partitioning

```
Partition 1: user_id 1 - 1,000,000
Partition 2: user_id 1,000,001 - 2,000,000
Partition 3: user_id 2,000,001 - 3,000,000
```

Pros: Range queries efficient, simple **Cons:** Hot spots (new users all go to last partition)

Hash-Based Partitioning

```
partition = hash(key) % num_partitions

user_id 12345 → hash(12345) % 16 → Partition 7
```

Pros: Even distribution **Cons:** Range queries require all partitions, resharding is painful

Directory-Based Partitioning

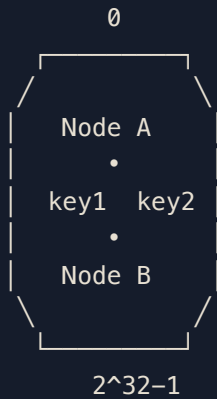
```
Lookup service:
user_id 12345 → Partition 3
user_id 67890 → Partition 1
```

Pros: Flexible, easy resharding **Cons:** Lookup service is bottleneck/SPOF

Consistent Hashing

Problem with $\text{hash}(\text{key}) \% N$: When N changes, almost all keys remap.

Solution: Consistent hashing maps keys and nodes to a ring.



Key placement: Walk clockwise from key's hash to first node

Adding a node: Only keys between predecessor and new node move ($\sim K/N$ keys) **Removing a node:** Only that node's keys redistribute

Virtual Nodes (VNodes)

Problem: Basic consistent hashing can be uneven (nodes get different arc sizes).

Solution: Each physical node owns multiple virtual nodes on the ring.

Physical Node A $\rightarrow$ vnodes: A1, A2, A3, A4, A5
Physical Node B $\rightarrow$ vnodes: B1, B2, B3, B4, B5
Physical Node C $\rightarrow$ vnodes: C1, C2, C3, C4, C5

150 vnodes per node (Cassandra default) $\rightarrow$ even distribution

Benefits:

- Even load distribution
- Heterogeneous hardware (more vnodes for powerful nodes)
- Gradual migration during rebalancing

Sharding in Practice

System	Strategy	Resharding
MongoDB	Range or hashed shard key	Manual (chunk migration)
Cassandra	Token ranges + vnodes	Automatic with vnodes
DynamoDB	Hash partition key	Managed by AWS
Vitess (MySQL)	Hash + lookup table	Reshard with dual-write
CockroachDB	Range-based	Automatic rebalancing

Hot Partition Problem

Symptoms: One shard gets disproportionate traffic (celebrity user, viral post)

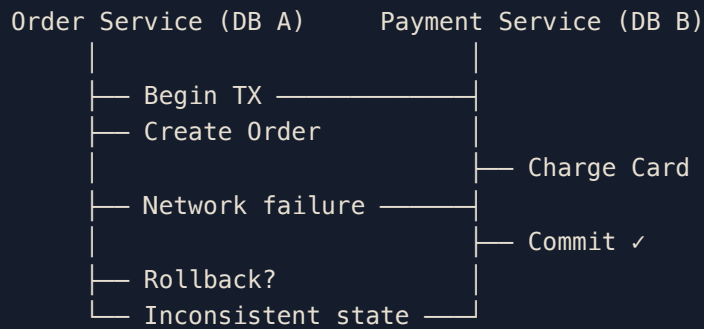
Solutions:

1. **Compound shard key:** `(user_id, post_id)` instead of just `user_id`
2. **Salting:** Append random suffix to spread writes
3. **Split hot partition:** Sub-partition the hot key range
4. **Caching layer:** Absorb read traffic on hot keys

7. Distributed Transactions

The Problem

Single-node ACID transactions don't work across services/databases:



Two-Phase Commit (2PC)

Coordinator-based protocol:

Phase 1 – Prepare:

Coordinator → All Participants: "Can you commit?"

Participants → Coordinator: "Yes (PREPARED)" or "No (ABORT)"

Phase 2 – Commit/Abort:

If all YES → Coordinator: "COMMIT"

If any NO → Coordinator: "ABORT"

Participants → Coordinator: "ACK"

Problems:

- **Blocking:** If coordinator fails after Phase 1, participants hold locks indefinitely
- **Single point of failure:** Coordinator crash = stuck transactions
- **Performance:** Two round trips + fsync on every participant
- **Not partition tolerant:** Network partition can cause blocking

Use When: Tight consistency required within a single organization's infrastructure (e.g., XA transactions in enterprise DBs). Avoid for microservices.

Saga Pattern

Choreography (Event-driven):

```
Order Service → OrderCreated event
↓
Payment Service → PaymentProcessed event
↓
Inventory Service → InventoryReserved event
↓
Shipping Service → ShipmentCreated event

Failure: Each service publishes compensating event
PaymentFailed → OrderCancelled (compensating action)
```

Orchestration (Central coordinator):

```
Saga Orchestrator:
1. Call OrderService.create()
2. Call PaymentService.charge()
3. Call InventoryService.reserve()
4. Call ShippingService.create()

On failure at step 3:
→ PaymentService.refund() (compensate)
→ OrderService.cancel() (compensate)
```

Comparison:

Aspect	Choreography	Orchestration
Coupling	Loose (events)	Centralized (orchestrator)
Visibility	Hard to trace	Clear workflow state
Complexity	Grows with services	Orchestrator can become complex
Best for	Simple flows (2-3 steps)	Complex flows (5+ steps)

Saga Rules:

- Each step must be **compensatable** (undo possible)
- Steps must be **idempotent** (safe to retry)
- No isolation guarantee (dirty reads possible between steps)

Outbox Pattern

Problem: How to atomically update DB and publish event?



Wrong:

```
db.transaction(() => {
  updateOrder(order);
  kafka.publish('order.created', order); // Can fail after commit!
});
```



Outbox Pattern:

```
db.transaction(() => {
  updateOrder(order);
  insertOutbox({ event: 'order.created', payload: order });
});
// Separate relay process reads outbox → publishes to Kafka → marks sent
```

Implementation:

```
// Step 1: Atomic write to business table + outbox table
async function createOrder(orderData) {
  await db.transaction(async (tx) => {
    const order = await tx.orders.create(orderData);
    await tx.outbox.create({
      aggregateType: 'Order',
      aggregateId: order.id,
      eventType: 'OrderCreated',
      payload: JSON.stringify(order),
      createdAt: new Date(),
    });
  });
}

// Step 2: Outbox relay (polling or CDC)
async function relayOutbox() {
  const pending = await db.outbox.findMany({
    where: { publishedAt: null },
    orderBy: { createdAt: 'asc' },
    take: 100,
  });

  for (const entry of pending) {
    await kafka.send({
      topic: entry.eventType,
      messages: [{ key: entry.aggregateId, value: entry.payload }],
    });
    await db.outbox.update({
      where: { id: entry.id },
      data: { publishedAt: new Date() },
    });
  }
}
```

CDC Alternative: Debezium reads database WAL/binlog → publishes to Kafka (no polling).

8. Idempotency & Exactly-Once Delivery

Idempotency

Definition: An operation produces the same result regardless of how many times it is executed.

$f(f(x)) = f(x)$

Examples:

SET x = 5 → Idempotent
x = x + 1 → NOT idempotent
DELETE WHERE id=1 → Idempotent
INSERT id=1 → NOT idempotent (duplicate key error)

Implementation Patterns:

```
// Idempotency key pattern
async function processPayment(idempotencyKey, paymentData) {
  const existing = await db.payments.findUnique({
    where: { idempotencyKey },
  });

  if (existing) {
    return existing; // Return cached result, don't re-process
  }

  const payment = await chargeCard(paymentData);
  await db.payments.create({
    idempotencyKey,
    ...payment,
  });
  return payment;
}
```

Idempotency Key Sources:

- Client-generated UUID (Stripe pattern)
- Hash of request parameters
- Natural key (order_id + action)

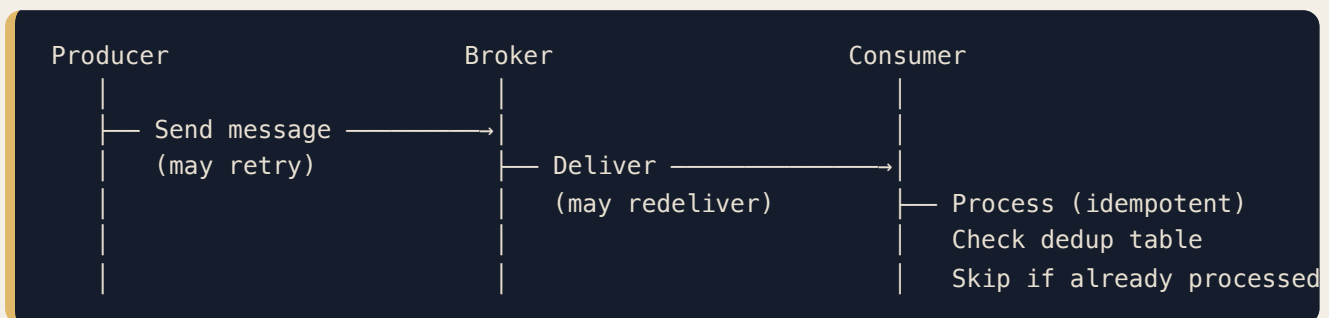
Delivery Semantics

Semantics	Meaning	Implementation
At-most-once	Message may be lost, never duplicated	Fire and forget
At-least-once	Message never lost, may be duplicated	Retry on failure + idempotent consumer
Exactly-once	Message processed exactly once	At-least-once + idempotency OR transactional

Exactly-Once Is a Lie (Mostly)

True exactly-once end-to-end is impossible in distributed systems. What we achieve:

Effective exactly-once = At-least-once delivery + Idempotent processing



Kafka Exactly-Once Semantics (EOS):

- Idempotent producer: `enable.idempotence=true` (PID + sequence numbers)
- Transactional producer: `transactional.id` + begin/commit/abort
- Read-process-write: Consumer reads → processes → writes output atomically

```
// Kafka transactional producer (conceptual)
const producer = kafka.proactiveProducer({
  transactionalId: 'order-processor-1',
  idempotent: true,
});

await producer.transaction(async (tx) => {
  const messages = await consumer.consume();
  const result = processOrder(messages);
  await tx.send({ topic: 'processed-orders', messages: result });
  await tx.sendOffsets({ consumerGroupId, topics: offsets });
});
```

9. Clocks: Physical, Logical, Vector

The Problem with Physical Clocks

```
Server A clock: 10:00:00.000
Server B clock: 10:00:00.050 (50ms ahead)
Server C clock: 09:59:59.980 (20ms behind)
```

Event on C at "09:59:59.990" may actually happen AFTER event on A at "10:00:00.000"
→ Cannot use wall clock for ordering events

NTP (Network Time Protocol): Synchronizes clocks but:

- Not perfect (milliseconds to seconds of drift)
- Clock jumps backward possible (leap seconds, corrections)
- Not suitable for ordering events

Google TrueTime (Spanner): GPS + atomic clocks, bounded uncertainty (~7ms). Uses commit wait to ensure ordering.

Lamport Logical Clocks

Rule: Each process maintains a counter. On event: increment. On send: attach counter. On receive: $\max(\text{local}, \text{received}) + 1$.

```

Process P1:  a(1) — b(2) — send m(3)
Process P2:                c(3) — recv m(4) — d(5)
Process P3:  e(1) — f(2)

```

If event A has timestamp < event B → A MAY have happened before B
 If event A has timestamp > event B → A definitely did NOT happen before B
 If equal → concurrent (cannot determine order)

Limitation: Cannot detect concurrency. If $L(A) < L(B)$, A might still be concurrent with B.

Vector Clocks

Each process maintains a vector of counters, one per process.

Initial: [0, 0, 0] (3 processes)

```

P1: event a → [1, 0, 0]
P1: send to P2 → [2, 0, 0]
P2: receive → [2, 1, 0]
P2: event b → [2, 2, 0]
P3: event c → [0, 0, 1]

```

Compare [2, 2, 0] vs [0, 0, 1]:
 Neither dominates → CONCURRENT events

Compare [2, 2, 0] vs [2, 1, 0]:
 [2,2,0] dominates → b happened after receive (causal)

Use Cases:

- Dynamo-style conflict detection
- Version vectors in Riak
- Causal consistency implementation

Hybrid Logical Clocks (HLC)

Combines physical and logical clocks (used in CockroachDB):

- Uses physical time when clocks are synchronized
- Falls back to logical counter when drift detected
- Provides bounded drift from physical time

10. Failure Modes

Network Partitions

Definition: Network failure splits cluster into groups that can't communicate.

Before partition: After partition:
A ↔ B ↔ C A ↔ B C (isolated)

CP system: C can't accept writes (no quorum)

AP system: C accepts writes, diverges from A-B

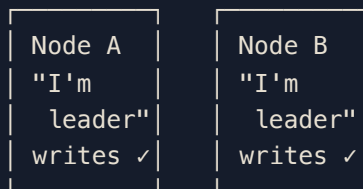
Detection: Heartbeats, gossip protocols, failure detectors

Recovery: Merge divergent state (anti-entropy, read repair, CRDT merge)

Split Brain

Definition: Two nodes both believe they are the leader, accepting writes independently.

Network Partition



→ Data divergence, corruption

Prevention:

1. **Quorum/fencing:** Require majority to be leader (Raft)
2. **Fencing tokens:** Monotonically increasing token; stale leader's writes rejected
3. **STONITH:** Shoot The Other Node In The Head — force shutdown of old leader
4. **Odd number of nodes:** Avoid 50-50 splits

```
// Fencing token pattern
async function writeWithFencing(key, value, fencingToken) {
  const currentToken = await lockService.getToken(key);
  if (fencingToken < currentToken) {
    throw new Error('Stale leader – fencing token rejected');
  }
  await db.update(key, value);
}
```

Byzantine Faults (Introduction)

Definition: Nodes may behave arbitrarily — crash, lie, or act maliciously.

Fault Type	Behavior	Example
Crash-stop	Node stops responding	Server crash
Omission	Node fails to send/receive	Network drop
Byzantine	Node sends conflicting info to different peers	Malicious node, bug

Byzantine Fault Tolerance (BFT):

- Requires $3f + 1$ nodes to tolerate f Byzantine failures
- PBFT (Practical Byzantine Fault Tolerance): Used in blockchain, Hyperledger
- Much more expensive than crash-fault tolerance ($2f + 1$ for Raft)

When It Matters:

- Blockchain/consensus (trustless environment)
- Multi-party systems with untrusted participants
- NOT typically needed in internal microservices (crash-fault is sufficient)

Common Failure Scenarios

Scenario	Impact	Mitigation
Leader crash	Write unavailability	Raft election (~150ms)
Follower crash	Reduced read capacity	Continue with remaining replicas
Network partition	Split brain risk	Quorum, fencing tokens
Slow node (straggler)	Increased latency	Read from followers, timeout
Cascading failure	Total system down	Circuit breakers, bulkheads, rate limiting
Data center failure	Regional outage	Multi-region replication

11. Interview Q&A (20 Questions)

Q1: Explain CAP theorem and give a real-world example of each choice.

Answer: CAP theorem states that in a distributed system experiencing a network partition, you must choose between Consistency and Availability — you cannot have both while maintaining Partition tolerance. Since network partitions are inevitable in distributed systems, P is always required in practice.

For **CP systems**, during a partition the system chooses consistency over availability. ZooKeeper is a classic example: if a quorum of nodes can't communicate, ZooKeeper will refuse writes rather than accept potentially inconsistent state. This is appropriate for coordination services where correctness is paramount — you don't want two nodes both thinking they're the leader.

For **AP systems**, during a partition the system remains available but may return stale data. Amazon DynamoDB (in its default configuration) continues serving reads and writes during partitions, but replicas may temporarily diverge. This suits shopping carts, social media feeds, and analytics where availability matters more than immediate consistency.

The key insight for senior interviews is that CAP isn't a one-time design choice — modern systems like Cassandra offer tunable consistency per query, and PACELC extends this to the latency-consistency trade-off during normal operation.

Q2: What is PACELC and how does it differ from CAP?

Answer: PACELC, proposed by Daniel Abadi in 2012, extends CAP theorem to cover the normal operating case when no network partition exists. The full statement is: "If there is a Partition (P), choose between Availability (A) and Consistency (C); Else (E), choose between Latency (L) and Consistency (C)."

CAP only tells you what happens during a partition. PACELC tells you what happens always. For example, Cassandra is PA/EL — it's available during partitions and prioritizes low latency over consistency during normal operation (you can get stale reads with consistency level ONE). Google Spanner is PC/EC — it maintains consistency even during partitions and uses TrueTime to provide consistent reads with bounded staleness during normal operation, at the cost of higher write latency.

This distinction matters in interviews because when designing a system, you must consider both scenarios. A payment system might need PC/EC (consistency always), while a recommendation engine might be fine with PA/EL (fast, eventually consistent).

Q3: Compare strong consistency, eventual consistency, and causal consistency.

Answer: These three models form a spectrum of consistency guarantees, each with different trade-offs.

Strong consistency (linearizability) guarantees that every read returns the most recent write. All operations appear to execute atomically in some sequential order consistent with real-time ordering. Implementation requires either a single leader with synchronous replication or quorum reads/writes where $R + W > N$. PostgreSQL with synchronous replication and etcd provide this. The cost is higher latency and reduced availability during failures.

Eventual consistency guarantees that if no new writes occur, all replicas will eventually converge to the same value. There is no bound on how long staleness persists. DynamoDB, Cassandra (at consistency level ONE), and DNS use this model. Conflict resolution via last-write-wins, vector clocks, or CRDTs is necessary when concurrent writes occur.

Causal consistency sits between them. If event A causally precedes event B (B reads or depends on A's write), all nodes see A before B. However, concurrent events (neither causally related) may be observed in different orders by different nodes. This is implemented using vector clocks. It's stronger than eventual but weaker than strong, suitable for social media comments/replies where you need to see a reply after its parent comment but don't need global ordering.

Q4: Explain Raft consensus algorithm. How does leader election work?

Answer: Raft is a consensus algorithm designed to be understandable while providing the same fault tolerance as Paxos. It decomposes consensus into leader election, log replication, and safety.

Leader election works as follows: Every node starts as a follower and waits for heartbeats from the current leader. If a follower doesn't receive a heartbeat within a randomized election timeout (typically 150-300ms), it transitions to candidate state, increments its term number, votes for itself, and sends RequestVote RPCs to all other nodes. A candidate becomes leader if it receives votes from a majority of nodes. If multiple candidates split the vote, each restarts its election timeout (randomized to reduce repeated splits) and tries again.

Key safety properties: At most one leader per term. A leader can only be elected if its log contains all committed entries (voters reject candidates with stale logs). Terms act as logical clocks — higher term always supersedes lower term.

Log replication: The leader accepts client requests, appends them to its log, and sends AppendEntries RPCs to followers. An entry is committed once replicated on a majority. The leader then applies committed entries to its state machine and responds to clients.

For production, run 3 or 5 nodes (odd number). Three nodes tolerate one failure; five tolerate two. etcd, Consul, and CockroachDB all use Raft variants.

Q5: Explain Paxos in simple terms. How does it relate to Raft?

Answer: Paxos is the original consensus algorithm, proven correct but notoriously difficult to understand and implement. It ensures that a group of nodes agree on a single value even if some nodes fail.

Paxos operates in two phases. In **Phase 1 (Prepare)**, a proposer sends a prepare request with a proposal number n to acceptors. Acceptors respond with a promise not to accept proposals numbered less than n , along with the highest-numbered proposal they've already accepted. In **Phase 2 (Accept)**, if the proposer receives promises from a majority, it sends an accept request with value v (using the highest accepted value from Phase 1 responses if any exist). If a majority accepts, the value is chosen.

Multi-Paxos optimizes this by electing a stable leader that skips Phase 1 for subsequent proposals, making it similar to Raft in practice.

The relationship: Raft was explicitly designed as an alternative to Paxos that is easier to understand and implement. Both guarantee safety (no two nodes decide different values) and liveness (eventually a value is decided) with up to f failures in $2f+1$ nodes. Raft achieves this through strong leader semantics; Paxos allows more flexibility but at the cost of complexity. In interviews, demonstrate understanding of Paxos conceptually but emphasize Raft for practical implementations.

Q6: Compare primary-replica, multi-leader, and leaderless replication.

Answer: These three replication topologies represent different trade-offs between simplicity, performance, and consistency.

Primary-replica (leader-follower): All writes go to a single primary node, which replicates to followers. Reads can go to primary (strong consistency) or followers (possible staleness). This is the simplest model — no write conflicts, straightforward failover. PostgreSQL, MySQL, and MongoDB use this. The downside is a write bottleneck on the primary and brief unavailability during leader election.

Multi-leader (multi-master): Multiple nodes accept writes, typically one leader per region or datacenter. Leaders replicate changes to each other asynchronously. This enables low-latency writes globally (write to nearest leader) and higher write availability. However, write conflicts are inevitable — the same record updated in two regions simultaneously requires conflict resolution (last-write-wins, custom merge logic, or CRDTs). Cassandra multi-datacenter and CouchDB use this model.

Leaderless (Dynamo-style): Any node accepts reads and writes. Consistency is controlled via quorum: with N replicas, W write acknowledgments and R read responses, if $W + R > N$, reads see the latest write. Cassandra and DynamoDB use this. Benefits include no leader election overhead and high availability. Downsides include complex conflict resolution, need for read repair to synchronize stale replicas, and sloppy quorum/hinted handoff complexity during node failures.

Choose primary-replica for simplicity and strong consistency, multi-leader for global low-latency writes, and leaderless for maximum availability with tunable consistency.

Q7: What is consistent hashing and why are virtual nodes needed?

Answer: Consistent hashing solves the problem of distributing keys across nodes in a way that minimizes remapping when nodes are added or removed.

Traditional hash partitioning uses $\text{hash}(\text{key}) \% N$. When N changes (node added/removed), nearly all keys remap to different nodes, causing massive data movement and cache invalidation.

Consistent hashing maps both nodes and keys onto a hash ring (0 to $2^{32}-1$). Each key is assigned to the first node encountered walking clockwise from the key's position. When a node is added, only keys between its predecessor and itself move. When removed, only its keys redistribute. Expected movement is K/N keys (K = total keys, N = nodes).

Virtual nodes (vnodes) solve the problem of uneven load distribution. With basic consistent hashing, if nodes are unevenly spaced on the ring, some nodes handle more keys than others. Each physical node is assigned multiple virtual nodes (typically 100-200) scattered around the ring. This ensures even distribution regardless of physical node count and allows weighting — a powerful machine can own more vnodes. Cassandra uses 256 vnodes per node by default; Dynamo uses 100-200.

In interviews, mention that consistent hashing is also used in load balancers, CDN routing, and distributed caches (Memcached clients).

Q8: Explain two-phase commit. What are its limitations?

Answer: Two-Phase Commit (2PC) is a distributed atomic commit protocol that ensures all participants either commit or abort a transaction together.

Phase 1 (Prepare/Voting): The coordinator sends a "prepare" message to all participants. Each participant writes a prepare record to its transaction log, acquires necessary locks, and responds "YES" (ready to commit) or "NO" (must abort). Participants remain in a prepared state with locks held.

Phase 2 (Commit/Abort): If all participants voted YES, the coordinator writes a commit decision to its log and sends "COMMIT" to all. Participants commit, release locks, and acknowledge. If any voted NO, the coordinator sends "ABORT" to all.

Critical limitations:

1. **Blocking:** If the coordinator crashes after Phase 1 but before Phase 2, participants are stuck holding locks indefinitely. They can't unilaterally commit (might be aborted) or abort (might be committed). This is the fundamental flaw.
2. **Single point of failure:** Coordinator failure halts all in-flight transactions.
3. **Performance:** Two synchronous round trips plus disk fsync on every participant for every transaction.
4. **Not partition tolerant:** A network partition between coordinator and a participant causes blocking.

For these reasons, 2PC is avoided in microservices architectures. Use Saga pattern or eventual consistency with outbox pattern instead. 2PC remains used in tightly coupled systems like XA transactions between colocated databases.

Q9: Explain the Saga pattern. When would you use choreography vs orchestration?

Answer: The Saga pattern manages distributed transactions as a sequence of local transactions, each with a corresponding compensating transaction for rollback.

Instead of a single ACID transaction spanning services, a saga breaks the operation into steps: create order → charge payment → reserve inventory → create shipment. Each step is a local transaction in its own service. If step 3 fails, compensating transactions undo steps 2 and 1 (refund payment, cancel order).

Choreography uses events — each service listens for events and reacts. OrderService publishes OrderCreated → PaymentService listens, charges, publishes PaymentProcessed → InventoryService listens, etc. On failure, PaymentService publishes PaymentFailed → OrderService listens and cancels. Benefits: loose coupling, no central coordinator. Drawbacks: hard to understand flow as it grows, difficult to track saga state, cyclic dependencies possible.

Orchestration uses a central saga orchestrator that tells each service what to do and tracks state. The orchestrator calls PaymentService.charge(), then InventoryService.reserve(), etc. On failure, it calls compensating actions in reverse order. Benefits: clear workflow visibility, centralized error handling, easier to add steps. Drawbacks: orchestrator can become a bottleneck and single point of failure.

Choose choreography for simple flows (2-3 services) where teams want independence. **Choose orchestration** for complex flows (5+ steps), when you need visibility into saga state, or when compensating logic is complex. In practice, orchestration with tools like Temporal or AWS Step Functions is more common at scale.

Key requirements: every step must be idempotent, compensating actions must be defined, and you accept that there's no isolation — other transactions may see intermediate states.

Q10: What is the outbox pattern and why is it needed?

Answer: The outbox pattern solves the dual-write problem: how to atomically update a

database and publish a message to a message broker without risking inconsistency.

The naive approach fails:

```
// ❌ NOT atomic – can fail between steps
await db.updateOrder(order);
await kafka.publish('order.created', order);
// If Kafka fails, DB is updated but no event published
// If DB fails after Kafka, event published but no DB update
```

The outbox pattern stores the event in the same database transaction as the business data:

```
// ✅ Atomic within DB transaction
await db.transaction(async (tx) => {
  await tx.orders.update(order);
  await tx.outbox.insert({ eventType: 'order.created', payload: order });
});
```

A separate **relay process** (polling or CDC) reads unpublished outbox entries, publishes them to Kafka, and marks them as published. Polling is simple but has latency (poll interval). CDC (Change Data Capture) using Debezium reads the database WAL/binlog for near-real-time propagation with no polling overhead.

This guarantees at-least-once delivery of events. Combined with idempotent consumers, you achieve effective exactly-once processing. The pattern is essential in event-driven microservices where service A must reliably notify service B about state changes. It's used by Uber, Netflix, and most mature microservices architectures.

Q11: How do you achieve idempotency in distributed systems?

Answer: Idempotency ensures that processing the same request multiple times produces the same result as processing it once. This is critical because networks cause retries, message brokers deliver at-least-once, and clients timeout and resend.

Implementation strategies:

1. **Idempotency keys:** The client sends a unique key (UUID) with each request. The server checks if this key was already processed — if yes, return the cached response without re-executing. Stripe and PayPal use this for payment APIs. Store keys with TTL (24-72 hours) in Redis or the database.

2. **Natural idempotency:** Design operations to be inherently idempotent. PUT /users/123 with body `{name: "Alice"}` is idempotent (same result every time). DELETE /users/123 is idempotent. POST /users (creates new) is NOT idempotent.
3. **Upsert with unique constraints:** Use database unique constraints so duplicate inserts fail gracefully. `INSERT INTO payments (idempotency_key, ...) ON CONFLICT (idempotency_key) DO NOTHING RETURNING *`
4. **State checks:** Before executing, check current state. "If order status is already PAID, return success without charging again."
5. **Version numbers/ETags:** Reject updates if the version doesn't match expected, preventing duplicate state transitions.

For message consumers, maintain a processed-messages table. Before processing, check if message ID exists. After processing, record the ID. This converts at-least-once delivery to exactly-once processing.

Q12: Explain at-least-once, at-most-once, and exactly-once delivery semantics.

Answer: These three semantics define how message delivery behaves in the presence of failures.

At-most-once: Messages are sent once with no retries. If delivery fails, the message is lost.

Implementation: fire-and-forget, no acknowledgment required. Use when data loss is acceptable — metrics, logs, non-critical notifications. Lowest overhead.

At-least-once: Messages are retried until acknowledged. Duplicates are possible if acknowledgment is lost after processing. Implementation: producer retries on timeout, broker redelivers if consumer doesn't ack, consumer processes and then acks. This is the most common guarantee because no data is lost. Requires idempotent consumers to handle duplicates safely.

Exactly-once: Each message is processed exactly one time. True end-to-end exactly-once is theoretically impossible (the Two Generals Problem). What's achievable in practice:

- **Kafka EOS:** Idempotent producer (PID + sequence numbers prevent duplicate writes) + transactional producer (atomic read-process-write across topics) + idempotent consumer with transactional offsets.
- **Effective exactly-once:** At-least-once delivery + idempotent processing + deduplication table.

In interviews, clarify that "exactly-once" in industry means "effectively exactly-once" through the combination of at-least-once delivery and idempotent processing. Ask what level of guarantee the

business requires — payment processing needs effective exactly-once; click tracking can tolerate at-most-once.

Q13: What are vector clocks and how do they detect conflicts?

Answer: Vector clocks are a mechanism for tracking causality across distributed events, enabling detection of concurrent (conflicting) writes.

Each process maintains a vector of N counters (one per process in the system). Rules:

- On local event: increment own counter in the vector
- On send: attach current vector to message
- On receive: element-wise max of local and received vector, then increment own counter

Comparison rules:

- V1 dominates V2 ($V1 > V2$) if every element of $V1 \geq V2$ and at least one is strictly greater → V1 happened after V2 (causal)
- V1 and V2 are **concurrent** if neither dominates the other → conflict detected

Example with 3 nodes:

```
P1 writes x=1: [1,0,0]
P2 receives, writes x=2: [1,1,0]
P3 writes y=3: [0,0,1]
```

```
Compare [1,1,0] and [0,0,1]: neither dominates → concurrent, no conflict (different key)
If P3 also wrote x=5: [0,0,1] vs [1,1,0] → concurrent writes to same key → CONFLICT
```

Conflict resolution options: last-write-wins (lose data), keep both (application merge), prompt user (shopping cart), or use CRDTs (automatic merge).

Vector clocks are used in Amazon Dynamo, Riak (version vectors), and anywhere causal consistency or conflict detection is needed. The downside is vector size grows with number of nodes — version vectors (track only nodes that have seen the object) mitigate this.

Q14: What is split brain and how do you prevent it?

Answer: Split brain occurs when a network partition causes two or more nodes to independently believe they are the leader, both accepting writes. This leads to data divergence

and corruption when the partition heals.

Classic scenario: 5-node cluster, partition splits into [A,B,C] and [D,E]. Without proper safeguards, both groups might elect a leader (C in first group, D in second), and clients connected to each group write different data to the same keys.

Prevention mechanisms:

1. **Quorum-based leadership:** Require majority ($N/2 + 1$) to elect a leader. In the example, only the [A,B,C] group (3/5) can elect a leader. The [D,E] group (2/5) cannot — it becomes read-only or unavailable.
2. **Fencing tokens:** The lock service provides monotonically increasing tokens with each leadership grant. Storage systems reject writes with stale tokens. Even if old leader doesn't know it lost leadership, its writes are rejected.
3. **STONITH (Shoot The Other Node In The Head):** Forcefully shut down or isolate the old leader using hardware/software fencing (IPMI, power off). Ensures old leader can't accept writes.
4. **Odd number of nodes:** With even nodes (4), a partition can split 2-2, and without external coordination, both sides might think they have quorum. Always use odd numbers (3, 5, 7).
5. **Lease-based leadership with TTL:** Leader must renew lease periodically. Short TTL ensures stale leaders stop quickly, but risks false failover if renewal is delayed.

In Kubernetes, etcd prevents split brain via Raft quorum. In Redis Sentinel, `min-slaves-to-write` and quorum configuration prevent split brain during failover.

Q15: How does quorum read/write work in Dynamo-style systems?

Answer: Quorum replication in leaderless systems like Dynamo and Cassandra provides tunable consistency without a single leader.

Configuration parameters:

- **N** = replication factor (number of replicas), typically 3
- **W** = write quorum (nodes that must acknowledge a write), typically 2
- **R** = read quorum (nodes queried for a read), typically 2

Strong consistency guarantee: If $W + R > N$, a read is guaranteed to overlap with the latest write. With $N=3$, $W=2$, $R=2$: write goes to 2 of 3 replicas. Read queries 2 of 3 replicas. By pigeonhole principle, at least one replica has the latest value.

Consistency levels in Cassandra:

Level	W/R	Behavior
ONE	1	Fastest, eventual consistency
QUORUM	majority	Strong consistency ($W+R>N$)
ALL	N	Slowest, strongest, unavailable if any node down

Sloppy quorum and hinted handoff: When a replica node is down, the coordinator stores a "hint" — it writes to a temporary node that will forward the data when the original node recovers. This maintains availability at the cost of temporary inconsistency.

Read repair: During a read, if replicas return different values, the coordinator returns the latest (by timestamp or vector clock) and asynchronously updates stale replicas.

In interviews, explain that quorum is a probabilistic consistency mechanism — it works well in practice but doesn't provide linearizability (concurrent reads and writes can still race). For true linearizability, you need a leader-based protocol like Raft.

Q16: What is the difference between horizontal and vertical partitioning?

Answer: Partitioning (sharding) splits data across multiple nodes. The two fundamental strategies are horizontal and vertical.

Horizontal partitioning (sharding) splits rows across nodes. Each partition contains a subset of rows but all columns. Example: users 1-1M on shard 1, users 1M-2M on shard 2. Or hash-based: `hash(user_id) % 16` determines shard. This is what people usually mean by "sharding." It scales throughput and storage. Challenges: cross-shard queries are expensive, resharding is complex, hot partitions possible.

Vertical partitioning splits columns across nodes. Each partition contains all rows but a subset of columns. Example: user profile (name, email) on one table/database, user activity (login history, preferences) on another. This optimizes for access patterns — frequently accessed columns together, rarely accessed columns separately. Also reduces row size for faster scans.

Vertical partitioning by service is common in microservices: User Service owns user data, Order Service owns order data. This is essentially vertical partitioning at the service level.

In practice, horizontal partitioning is used for scaling (more common in interviews), while vertical partitioning is used for optimization and microservice boundaries. Many systems use both: vertical split by domain (microservices), horizontal split within each service (sharding).

Q17: How would you handle a hot partition in a sharded database?

Answer: A hot partition (hot shard) occurs when one partition receives disproportionate traffic compared to others, becoming a bottleneck despite having multiple shards.

Common causes:

- Celebrity/viral content (millions accessing one user's data)
- Poor shard key choice (timestamp-based keys funnel recent data to one shard)
- Sequential ID generation (all new records go to the last shard in range partitioning)

Detection: Monitor per-shard QPS, CPU, and storage metrics. Alert when any shard exceeds 2-3x the average.

Solutions:

1. **Better shard key:** Use a compound key that distributes load. Instead of sharding tweets by `user_id` (celebrity problem), shard by `(user_id, tweet_id)` or add a random suffix bucket.
2. **Salting:** Append a random suffix to the shard key for hot entities. `celebrity_user_id + random(0-9)` spreads one user's data across 10 shards. Reads must query all salted shards (trade-off).
3. **Split the hot partition:** Divide the key range of the hot shard into sub-shards. MongoDB automatically splits chunks when they exceed size threshold.
4. **Caching layer:** Place Redis/Memcached in front of hot keys. Absorb 90%+ of read traffic before it hits the shard. This is often the fastest fix.
5. **Rate limiting:** Throttle requests to hot keys to prevent cascade failure while implementing a permanent fix.
6. **Async processing:** For write-heavy hot keys (like counters), buffer writes in memory/queue and batch-flush periodically.

At 80 LPA level, discuss the trade-offs: salting improves write distribution but complicates reads; caching is fast but adds consistency concerns; splitting is clean but requires rebalancing infrastructure.

Q18: What are Byzantine faults and when do they matter?

Answer: Byzantine faults occur when a node behaves arbitrarily — it may crash, omit messages, or actively send different/conflicting information to different peers. Named after the Byzantine

Generals Problem.

Fault hierarchy:

- **Crash-stop:** Node fails and stops responding (most common, handled by Raft/Paxos)
- **Omission:** Node fails to send or receive messages (network issues)
- **Byzantine:** Node behaves maliciously or has bugs causing it to lie (sends different values to different nodes)

Byzantine Fault Tolerance (BFT) requires $3f + 1$ nodes to tolerate f Byzantine failures (vs $2f + 1$ for crash faults). PBFT (Practical Byzantine Fault Tolerance) is the classic algorithm, requiring $O(n^2)$ messages per operation.

When Byzantine faults matter:

- Blockchain and cryptocurrency (trustless environment — any node could be malicious)
- Multi-organization systems where participants don't trust each other
- Safety-critical systems where a bug could cause a node to send conflicting data

When they DON'T matter (most systems):

- Internal microservices (you control all nodes; crash faults suffice)
- Database replication (nodes fail, they don't lie)
- Cloud infrastructure (trust your cloud provider's nodes)

For 80 LPA interviews, demonstrate awareness of Byzantine faults but correctly scope them — "For our internal payment system, crash fault tolerance with Raft is sufficient. We'd need BFT only if we were building a cross-bank settlement network where participants aren't trusted."

Q19: Design a distributed lock service. What guarantees would you provide?

Answer: A distributed lock service provides mutual exclusion across processes/nodes in a distributed system.

Requirements:

- **Safety:** At most one holder at any time
- **Liveness:** Lock eventually acquired by some requester
- **Fault tolerance:** Survives node failures
- **Fencing:** Prevent stale lock holders from writing

Design using etcd/Redis:

Acquire:

1. Create ephemeral sequential node: `/locks/my-resource/000000001`
2. Get all children of `/locks/my-resource`
3. If my node is smallest → I hold the lock
4. Else → watch the node before me, goto 2 on deletion

Release:

1. Delete my node
2. Next waiter is notified via watch

Using Redis (Redlock algorithm — controversial):

1. Get timestamp T1
2. Try SET NX EX on N independent Redis instances (majority must succeed)
3. Lock valid if acquired on majority AND $(T2 - T1) < TTL$
4. Release with Lua script (check value matches before delete)

Critical: Fencing tokens. A stale lock holder (GC pause caused it to think it still holds the lock) can corrupt data. Solution: the lock service returns a monotonically increasing fencing token. The storage layer rejects writes with tokens older than the highest seen.

```
async function doWorkWithLock(resource) {
  const { acquired, fencingToken } = await lockService.acquire(resource, { ttl: 30 });
  if (!acquired) throw new Error('Could not acquire lock');

  try {
    await storage.write(resource, data, { fencingToken });
  } finally {
    await lockService.release(resource, fencingToken);
  }
}
```

Guarantees to offer: Mutual exclusion (with fencing), deadlock freedom (TTL-based auto-release), fault tolerance (quorum-based). Do NOT claim exactly-once — locks provide at-most-one execution with fencing for safety.

Q20: How does Google Spanner achieve global consistency?

Answer: Google Spanner achieves external consistency (linearizability across globally distributed datacenters) through a combination of TrueTime, Paxos-based replication, and two-phase commit.

TrueTime API: Returns an interval [earliest, latest] representing current time with bounded uncertainty ($\sim 7\text{ms}$). TrueTime uses GPS receivers and atomic clocks in every datacenter. This bounded uncertainty is the key enabler.

Write path:

1. Client sends write to Spanner leader (per shard)
2. Leader assigns timestamp using TrueTime (commit timestamp = `TT.now().latest`)
3. Leader replicates via Paxos to other datacenters
4. **Commit wait:** Leader waits until `TT.now().earliest > commit timestamp` before acknowledging. This ensures no subsequent transaction can get an earlier timestamp, maintaining global ordering.
5. Respond to client

Read path:

- Strong reads use a timestamp and wait for `TT.now().earliest > read timestamp` (no stale reads)
- Staleness-bound reads can skip commit wait for lower latency

Distributed transactions: Spanner uses two-phase commit across shard leaders, with Paxos replication at each shard. TrueTime provides global transaction ordering.

Trade-offs: Commit wait adds $\sim 7\text{ms}$ latency to writes (the uncertainty bound). Spanner chooses PC/EC in PACELC terms — consistency always, even at the cost of latency. This is appropriate for Google's use cases (AdWords billing, Play Store) where correctness is paramount.

For interviews, contrast with DynamoDB (PA/EL — fast but eventually consistent) to show you understand the design space and can choose appropriately for different requirements.

12. Where to Practice

Books

- **"Designing Data-Intensive Applications"** by Martin Kleppmann — The bible for distributed systems interviews
- **"Database Internals"** by Alex Petrov — Deep dive into storage and replication
- **"Understanding Distributed Systems"** by Roberto Valerio — Concise, practical

Online Resources

- [Jepsen.io](#) — Distributed systems correctness testing (read analyses)
- [Raft Consensus Visualization](#) — Interactive Raft demo
- [ByteByteGo Newsletter](#) — System design concepts
- [MIT 6.824 Distributed Systems](#) — Lecture notes and labs

Papers (Read Summaries First)

- Dynamo (Amazon) — Leaderless, consistent hashing, eventual consistency
- Google Spanner — TrueTime, global consistency
- Raft paper — Understandable consensus
- Google MapReduce / GFS — Foundational distributed computing

Hands-On Practice

- Deploy etcd cluster locally, kill nodes, observe leader election
- Set up Cassandra cluster, test different consistency levels
- Implement a simple Raft leader election simulation
- Use Jepsen-style testing: kill nodes during writes, verify consistency

Interview Platforms

- [Exponent](#) — System design mock interviews
- [Hello Interview](#) — FAANG system design prep
- [System Design Interview \(Volume 1 & 2\)](#) by Alex Xu

Key Topics to Drill

1. Explain CAP/PACELC with specific system examples
2. Walk through Raft leader election on a whiteboard
3. Design a distributed cache with consistent hashing
4. Compare Saga vs 2PC for a payment flow
5. Explain how you'd achieve exactly-once processing in an event-driven system

Last updated: August 2026 | Target: 80+ LPA SDE (Google L5, Meta E4, Amazon SDE3, Microsoft L63+)

Distributed Systems Interview – 50 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

This bank covers consistency models, replication, consensus algorithms, and the saga pattern for distributed systems interviews at product companies. Spell out abbreviations on first use.

1. What is a distributed system?

A distributed system is a collection of independent computers that work together and appear to users as a single coherent system. Each node has its own memory and may fail independently. Examples include Google Search, Amazon DynamoDB, and Netflix microservices. The main challenge is coordinating nodes over an unreliable network while maintaining correctness and performance.

2. What is replication in distributed databases?

Replication means keeping copies of the same data on multiple nodes. It improves read throughput, provides fault tolerance, and reduces latency by serving from nearby replicas. Trade-offs include consistency — replicas may be temporarily out of sync — and write complexity because updates must propagate to all copies.

3. What is leader-based replication?

In leader-based (primary-secondary) replication, one node is the leader that accepts all writes. Follower replicas receive a stream of changes from the leader and apply them locally. Reads can go to the leader for strong consistency or followers for eventual consistency. PostgreSQL and MySQL use this model; failover promotes a follower when the leader dies.

4. What is synchronous vs asynchronous replication?

Synchronous replication waits for follower acknowledgment before confirming a write — guarantees no data loss on failover but increases write latency. Asynchronous replication confirms immediately

after the leader writes locally; followers catch up later — faster writes but recent data may be lost if the leader fails before replication completes. Many systems offer configurable sync levels.

5. What is multi-leader replication?

Multi-leader replication allows writes on multiple nodes, which then replicate to each other. Useful for multi-datacenter deployments where each region writes locally. Challenges include write conflicts when two leaders modify the same record. Resolution strategies include last-write-wins, custom merge logic, or partitioning so each leader owns a key range.

6. What is leaderless replication?

Leaderless replication (used by Amazon DynamoDB and Apache Cassandra) allows any node to accept reads and writes. Writes go to multiple replicas; reads query multiple replicas and use version numbers to resolve conflicts. Quorum reads and writes ensure consistency when enough nodes agree. No single leader means no leader failover bottleneck, but conflict resolution is the caller's responsibility.

7. What is the quorum formula $R + W > N$?

In a system with N replicas, R is the number of nodes read from and W is the number written to. The quorum condition $R + W > N$ ensures read and write sets overlap, so reads see the latest write. Example: $N=3$, $W=2$, $R=2$ guarantees strong consistency. Lower W improves write speed; lower R improves read speed — but if $R + W \leq N$, stale reads are possible.

8. What is linearizability (strong consistency)?

Linearizability means every operation appears to take effect at a single instant between its start and end, and all nodes agree on the order. If operation A completes before B starts, B must see A 's effects. This is the strongest single-object consistency guarantee. Required for distributed locks, leader election, and financial balances.

9. What is sequential consistency?

Sequential consistency guarantees all nodes see the same order of operations, but that order may differ from real-time order. If two writes happen concurrently with no causal link, different nodes

might observe them in different orders — but each individual node sees a consistent sequence. Weaker than linearizability but easier to implement in some systems.

10. What is causal consistency?

Causal consistency preserves cause-and-effect relationships: if operation B reads or depends on operation A, all nodes must see A before B. Concurrent operations with no causal link may be seen in different orders. Implemented using vector clocks or version vectors. Stronger than eventual consistency, weaker than sequential consistency — good for social media comments and collaborative editing.

11. What is read-your-writes consistency?

Read-your-writes means after a user writes data, they always see their own update on subsequent reads — even if other users might see stale data temporarily. Achieved by routing a user's reads to the node they wrote to, or by tracking write timestamps and rejecting stale reads. Essential for user experience in profile updates and shopping carts.

12. What is monotonic reads consistency?

Monotonic reads guarantee that if a user reads a value, they will never see an older value on subsequent reads (no time travel backward). Even if replicas are temporarily inconsistent, a user is pinned to a replica or reads are version-checked. Prevents confusing user experience like a posted comment disappearing and reappearing.

13. What is eventual consistency?

Eventual consistency means if no new updates occur, all replicas will converge to the same value over time. Writes propagate asynchronously; reads may return stale data temporarily. Acceptable for social like counts, view counts, and product recommendations. Amazon DynamoDB and Apache Cassandra default to this model. Mitigate with version numbers and user-facing sync indicators.

14. What is a vector clock?

A vector clock tracks causality across distributed nodes. Each node maintains a vector of counters (one per node). On a local event, increment your counter; on sending a message, attach your vector;

on receiving, merge by taking the max of each element. Comparing vectors reveals whether events are concurrent or causally ordered — essential for conflict detection in multi-leader systems.

15. What is the Two-Phase Commit (2PC) protocol?

Two-Phase Commit (2PC) is a distributed transaction protocol ensuring all participants commit or all abort. Phase 1 (Prepare): coordinator asks all participants to vote; each prepares and responds yes/no. Phase 2 (Commit/Abort): if all voted yes, coordinator sends commit; otherwise abort.

Problem: coordinator failure after prepare leaves participants blocked — requires manual intervention or timeout.

16. What is the Three-Phase Commit (3PC) protocol?

Three-Phase Commit (3PC) adds a pre-commit phase to 2PC to reduce blocking. Phase 1: CanCommit (probe). Phase 2: PreCommit (prepare). Phase 3: DoCommit. If the coordinator fails after pre-commit, participants can safely commit because all agreed to proceed. However, 3PC still fails under network partitions and is rarely used in production — saga pattern is preferred.

17. What is the saga pattern?

The saga pattern manages distributed transactions as a sequence of local transactions, each with a compensating action if a later step fails. Instead of locking across services (like 2PC), each step commits independently. If step 3 fails, run compensating transactions for steps 2 and 1 (refund payment, release inventory). Used heavily in microservices for order processing and booking systems.

18. What is saga orchestration vs choreography?

Saga orchestration uses a central coordinator that tells each service what to do and handles failures. Easier to understand and debug but creates a single point of logic. Saga choreography has each service listen for events and decide locally (payment completed → shipping service acts). More decoupled but harder to trace. Choose orchestration for complex flows; choreography for simple event chains.

19. What is a compensating transaction?

A compensating transaction undoes the effect of a previously committed local transaction in a saga. It is not a database rollback — that window is closed — it is a new business operation: refund a charge, cancel a reservation, send a cancellation email. Compensating actions must be idempotent because they may be retried. Design them during saga planning, not as an afterthought.

20. What is the Raft consensus algorithm?

Raft is a consensus algorithm for electing a leader and replicating a log across nodes. States: leader, follower, candidate. Leader accepts client requests, appends to its log, replicates to followers; commit when majority acknowledges. On leader failure, followers timeout and start an election. Easier to understand than Paxos; used by etcd, Consul, and CockroachDB. Guarantees committed entries are never lost.

21. What is the Paxos consensus algorithm?

Paxos is the foundational consensus algorithm proving that a distributed system can agree on a value despite failures. Roles: proposers, acceptors, learners. Uses proposal numbers and majority quorums. Correct but notoriously difficult to implement and explain. Variants like Multi-Paxos optimize for continuous log replication. Google Chubby and early Apache ZooKeeper implementations use Paxos-family algorithms.

22. What is distributed consensus and why is it needed?

Distributed consensus means multiple nodes agreeing on a single value or sequence of values despite failures and network delays. Needed for leader election, atomic commit, and replicated state machines (same log order on all nodes). Without consensus, split-brain and data corruption occur. Consensus requires a majority (quorum) to tolerate f failures in $2f+1$ nodes.

23. What is split-brain in distributed systems?

Split-brain occurs when a network partition causes two groups of nodes to each believe they are the primary, accepting writes independently. Data diverges and cannot be automatically merged. Prevention: require majority quorum for leadership, use fencing tokens to reject stale leaders, and deploy an odd number of nodes across failure domains.

24. What is a fencing token?

A fencing token is a monotonically increasing number issued by the lock service (Apache ZooKeeper, etcd) when a client acquires a lock. The client must present this token with every write; storage rejects writes with stale (lower) tokens. Prevents a delayed old leader from corrupting data after losing leadership. Critical for safe distributed locks in Apache Kafka and Google Chubby.

25. What is exactly-once semantics?

Exactly-once semantics means each message or operation is processed precisely one time with no duplicates and no losses. Extremely difficult in distributed systems — network retries cause duplicates; failures cause losses. Practical approach: at-least-once delivery plus idempotent consumers (deduplicate by message ID). Apache Kafka transactions and idempotent producers approximate exactly-once within the Kafka ecosystem.

26. What is at-least-once vs at-most-once delivery?

At-least-once delivery guarantees every message is delivered one or more times — retries on failure — so consumers must handle duplicates. At-most-once delivery delivers zero or one time (no retries) — messages may be lost but never duplicated. Exactly-once is the ideal but hardest. Most production systems use at-least-once with idempotent processing for reliability.

27. What is idempotency in distributed systems?

Idempotency means performing an operation multiple times has the same effect as performing it once. Critical for safe retries after timeouts. Implement via unique request IDs stored in a database: first request processes and records the ID; duplicates return the cached result. Payment systems, reservation Application Programming Interfaces (APIs), and message consumers all require idempotent design.

28. What is a distributed transaction?

A distributed transaction spans multiple services or databases and must commit or abort atomically across all participants. Classic approach: Two-Phase Commit (2PC) — simple in theory but slow, blocking, and fragile. Modern approach: saga pattern with compensating transactions. Avoid distributed transactions when possible — design services to own their data and communicate via events.

29. What is the difference between ACID and BASE?

ACID (Atomicity, Consistency, Isolation, Durability) describes traditional relational database guarantees — strong consistency, used in PostgreSQL and MySQL. BASE (Basically Available, Soft state, Eventual consistency) describes many NoSQL systems — prioritize availability and partition tolerance, accept temporary inconsistency. Distributed systems often choose BASE at scale and enforce strong consistency only where business requires it (payments).

30. What is a gossip protocol?

A gossip protocol spreads information like epidemic rumor spreading: each node periodically sends its state to random peers, which forward to others. Eventually all nodes converge. Used for membership detection, spreading cluster metadata, and anti-entropy repair in Amazon DynamoDB and Apache Cassandra. Highly scalable and fault-tolerant but eventually consistent with propagation delay.

31. What is consistent hashing in distributed systems?

Consistent hashing maps both data keys and nodes onto a hash ring. Adding or removing a node only remaps keys adjacent to that node — unlike modulo hashing ($\text{key} \% N$) which remaps almost everything. Virtual nodes (multiple points per physical node) improve load balance. Used in distributed caches (Memcached), databases (Cassandra), and load balancers.

32. What is sharding vs partitioning?

Partitioning divides data into subsets within one database (horizontal partitioning by key range or hash). Sharding distributes partitions across multiple independent database instances (shards). All sharding involves partitioning, but not all partitioning is sharding. Sharding enables horizontal write scaling beyond single-node limits. Challenges: cross-shard queries, rebalancing, and hot shards.

33. What is a hot shard problem?

A hot shard occurs when one shard receives disproportionate traffic because its key range includes a popular entity (celebrity user, viral product, global counter). That shard becomes a bottleneck while others sit idle. Mitigations: salt hot keys (append random suffix to spread writes), separate hot data to dedicated infrastructure, use local cache in front of hot shard, or redesign key distribution.

34. What is clock synchronization and why does it matter?

Computers have independent clocks that drift apart. Distributed systems use Network Time Protocol (NTP) or Precision Time Protocol (PTP) to align timestamps. Matters for conflict resolution (last-write-wins), event ordering, and debugging. Google Spanner uses TrueTime (GPS plus atomic clocks) for globally consistent timestamps. Never assume perfect clock sync — design for clock skew.

35. What is a Lamport timestamp?

A Lamport timestamp is a logical clock: each node has a counter incremented before each event; on receiving a message, set counter to $\max(\text{local}, \text{received}) + 1$. If event A's timestamp is less than B's, A might have happened before B (but not guaranteed if concurrent). Establishes partial ordering without physical clocks. Foundation for vector clocks and distributed debugging.

36. What is Byzantine fault tolerance?

Byzantine fault tolerance (BFT) handles nodes that behave arbitrarily — lying, corrupting data, or acting maliciously — not just crashing. Requires $3f+1$ nodes to tolerate f Byzantine failures. Used in blockchain consensus (Proof of Work, Practical Byzantine Fault Tolerance — PBFT) and critical infrastructure. Most business systems assume crash-stop failures only, which requires only $2f+1$ nodes.

37. What is a distributed lock and how is it implemented?

A distributed lock ensures only one process accesses a shared resource at a time across multiple nodes. Implementations: Redis SET NX with expiry, Apache ZooKeeper ephemeral sequential nodes, etcd leases. Must handle lock holder crashes (use Time To Live expiry), clock skew (use fencing tokens), and network partitions (require quorum). Redlock algorithm for Redis is debated — prefer etcd or ZooKeeper for correctness-critical locks.

38. What is Apache ZooKeeper used for?

Apache ZooKeeper is a coordination service providing a hierarchical key-value store with strong consistency, ephemeral nodes, and watches. Used for leader election, distributed locks, configuration management, and service discovery. Clients maintain sessions; ephemeral nodes

disappear on session expiry (detect failures). Apache Kafka historically used ZooKeeper; modern Kafka uses KRaft (Kafka Raft) mode instead.

39. What is etcd?

etcd is a distributed key-value store using the Raft consensus algorithm, developed by CoreOS (now Red Hat). Kubernetes uses etcd as its backing store for all cluster state (pods, services, configs). Provides strong consistency, watch notifications, and lease-based Time To Live (TTL). Simpler than ZooKeeper for cloud-native systems. Always run etcd with odd-numbered node count for quorum.

40. What is the difference between horizontal and vertical partitioning?

Vertical partitioning splits a table by columns — frequently accessed columns in one partition, rarely accessed in another (reduces input/output). Horizontal partitioning splits by rows using a partition key (user ID, date range) — enables sharding across machines. Horizontal partitioning is what people usually mean by sharding. Choose partition key based on query patterns to avoid cross-partition joins.

41. What is read repair?

Read repair is a consistency mechanism in leaderless systems: during a read, if replicas return different versions, the coordinator writes the latest version back to stale replicas before returning to the client. Keeps replicas converging without a dedicated repair process. Amazon DynamoDB and Apache Cassandra use read repair alongside hinted handoff and anti-entropy to maintain consistency.

42. What is anti-entropy in distributed systems?

Anti-entropy is a background process that compares and synchronizes data between replicas to fix divergence. Apache Cassandra runs Merkle tree comparisons between nodes — if hashes differ, exchange differing data ranges. Runs periodically (not on every read). Complements read repair and hinted handoff. Essential for long-term consistency in eventually consistent systems.

43. What is a Merkle tree in replication?

A Merkle tree is a hash tree where leaf nodes are hashes of data blocks and parent nodes are hashes of their children. Comparing root hashes quickly detects if two replicas differ; if different, compare child hashes to pinpoint differing ranges without transferring all data. Used in anti-entropy repair (Cassandra, Bitcoin) and peer-to-peer file systems (BitTorrent, InterPlanetary File System).

44. What is the difference between failover and switchover?

Switchover is a planned transfer of primary role to another node (maintenance, upgrades) — controlled and graceful. Failover is unplanned transfer due to primary failure — automatic detection and promotion. Failover risks: data loss (async replication), split-brain (without quorum), and brief unavailability during election. Design for automatic failover with health checks and odd-numbered quorum nodes.

45. What is a distributed tracing span?

A span represents a single unit of work in a distributed trace (one service call, one database query). Spans have a trace ID (shared across the request), span ID, parent span ID, start/end timestamps, and metadata (service name, status). Spans form a tree showing the full request path. Tools: Jaeger, Zipkin, Amazon Web Services X-Ray. Essential for debugging latency in microservices.

46. What is the circuit breaker pattern in distributed systems?

The circuit breaker prevents cascade failures by stopping calls to a failing downstream service. States: Closed (normal), Open (fail fast after threshold failures), Half-Open (allow probe requests to test recovery). Returns fallback response when open. Libraries: Resilience4j, Istio service mesh. Pair with timeouts, retries with backoff, and bulkheads for resilient distributed systems.

47. What is the bulkhead pattern?

The bulkhead pattern isolates resources so failure in one area does not exhaust shared resources. Named after ship compartments that contain flooding. Example: separate thread pools for payment service calls vs search service calls — if search is slow, payment still has dedicated threads. In Kubernetes, separate deployments and resource limits per service achieve bulkhead isolation.

48. What is backpressure in distributed systems?

Backpressure is a flow control mechanism where a downstream component signals upstream to slow down when overwhelmed. Without backpressure, buffers overflow and systems crash.

Implementations: bounded queues that reject when full, rate limiting, and reactive streams (Project Reactor, Akka). Apache Kafka consumers control backpressure via fetch rate and processing speed; slow consumers increase consumer lag.

49. What is the difference between orchestration and choreography?

Orchestration has a central coordinator directing each service step-by-step (saga orchestrator, workflow engine like Temporal). Choreography has services react to events independently without a central brain (OrderCreated event → payment service, inventory service each act). Orchestration: easier to monitor and debug. Choreography: looser coupling, no single point of failure. Many systems combine both.

50. How do you test distributed systems?

Test at multiple levels: unit tests for individual components; integration tests with real message queues and databases; chaos engineering (deliberately kill nodes, inject latency with tools like Chaos Monkey, Litmus); property-based testing for consensus invariants; load testing for partition behavior under stress. Use deterministic simulation (Jepsen framework) to find consistency bugs. Monitor production with synthetic transactions and alerting on anomaly patterns.